

Universidad San Carlos de Guatemala
Facultad de ingeniería.
Ingeniería en ciencias y sistemas



MarkdownCheck: Analizador Léxico-Sintáctico de Markdown a HTML

PONDERACIÓN: 45 pts

Tiempo estimado: 35 hrs

Índice

1. MARCO FORMATIVO	3
1.1. Valor	3
1.2. Competencia(s)	3
1.3. Habilidad(es) blandas a formar	3
2. Resultado del Aprendizaje	3
2.1. Objetivo SMART	3
3. Resumen Ejecutivo	4
4. Enunciado del Proyecto	4
4.1 Descripción del problema o necesidad a resolver	4
4.2 Alcance del proyecto	4
4.3 Recursos y herramientas a utilizar	4
4.4 Entregables	4
5. Material de apoyo	5
6. Metodología	5
7. Cronograma	6
8. Rúbrica de Calificación	6
8.1 Requisitos para optar a la calificación	6
8.2 Detalle de la Calificación	7
8.3 Comentarios Generales	8

1. MARCO FORMATIVO

1.1. Valor

Nombre del valor	¿Cómo se aplica en el proyecto?
Rigor	El análisis sintáctico exige diseñar una gramática libre de contexto formalmente correcta y un parser que la respete al pie de la letra. Un solo descuido en una producción o en el manejo de la recursión provoca resultados incorrectos silenciosos, por lo que este proyecto exige del estudiante disciplina y precisión en cada decisión de diseño.

1.2. Competencia(s)

Con la elaboración de este proyecto usted adquirirá la siguiente competencia:

Competencia General: Diseñar, implementar y evaluar soluciones computacionales aplicando los fundamentos teóricos y prácticos de las ciencias de la computación para resolver problemas reales en el ámbito del desarrollo de software. Competencia Específica: Aplicar los conceptos de gramáticas libres de contexto y análisis sintáctico descendente recursivo para desarrollar, en Python, un analizador léxico-sintáctico que reconozca la estructura de documentos Markdown y genere su representación equivalente en HTML, implementando el parser manualmente sin librerías de parsing ni bibliotecas de conversión Markdown existentes.

1.3. Habilidad(es) blandas a formar

El proyecto le permitirá desarrollar las siguientes habilidades:

Autogestión del tiempo: planificar las 4 fases del proyecto a lo largo de aproximadamente un mes, dedicando entre 8 y 9 horas semanales.

Pensamiento abstracto: traducir reglas de un lenguaje natural ("una lista puede contener sublistas") a producciones formales de una gramática libre de contexto y a código recursivo.

Atención al detalle: resolver correctamente las ambigüedades propias de Markdown (por ejemplo, un asterisco que puede ser viñeta de lista o delimitador de cursiva según el contexto).

Comunicación técnica: documentar con claridad la gramática definida, las decisiones de diseño del parser y la estrategia de recuperación de errores en el manual técnico.

2. Resultado del Aprendizaje

2.1. Objetivo SMART

Específico (¿Qué?)	Medible (¿Cuánto?)	Alcanzable (¿Cómo?)	Realista (¿Para qué?)	A Tiempo (¿Cuándo?)
Los estudiantes diseñarán e implementarán desde cero, sin librerías de parsing ni de conversión Markdown, un analizador léxico-sintáctico en Python con interfaz gráfica, capaz de reconocer la gramática libre de contexto de un subconjunto de Markdown y traducirlo a HTML equivalente.	El analizador reconocerá correctamente al menos el 90% de los elementos Markdown definidos en los casos de prueba, incluyendo listas y citas anidadas en al menos 3 niveles de profundidad, y generará 3 reportes HTML exportables además del árbol de derivación en Graphviz.	Utilizando Python 3.10+ con una interfaz gráfica (Tkinter u otra librería de GUI en Python), AFD implementado manualmente para el análisis léxico y un parser descendente recursivo implementado manualmente para el análisis sintáctico, Git/GitHub para control de versiones y Graphviz para el árbol de derivación.	Fortalece las competencias en gramáticas libres de contexto, parsers descendentes recursivos y recuperación de errores sintácticos, cerrando el ciclo de análisis léxico (Proyecto 1) con el análisis sintáctico completo.	Completar en aproximadamente 4 semanas: Semana 1 diseño de la gramática y del AFD, Semana 2 implementación del analizador léxico, Semana 3 implementación del parser y la GUI, Semana 4 generación de reportes, pruebas y documentación.

3. Resumen Ejecutivo

MarkdownCheck es un proyecto de desarrollo de software cuyo objetivo es construir, en Python, un analizador léxico-sintáctico con interfaz gráfica capaz de leer, interpretar y traducir documentos de texto escritos en un subconjunto del lenguaje Markdown hacia su equivalente en HTML. El proyecto se enfoca en la implementación manual de un Autómata Finito Determinista (AFD) para el análisis léxico y de un parser descendente recursivo para el análisis sintáctico, ambos basados en una gramática libre de contexto definida formalmente, sin recurrir a librerías de parsing ni a bibliotecas de conversión de Markdown ya existentes.

A diferencia del Proyecto 1, donde el análisis se limitaba a reconocer tokens de forma lineal, aquí la dificultad central está en el anidamiento: una lista puede contener sublistas, una cita puede contener otra cita, y el texto en línea puede combinar negrita y cursiva simultáneamente. Esto obliga al estudiante a construir un verdadero árbol de derivación y a recorrerlo para generar el HTML de salida, en lugar de traducir línea por línea. La solución integra el motor léxico, el parser sintáctico, la interfaz gráfica, el generador de tres reportes HTML y el módulo de visualización del árbol de derivación con Graphviz.

4. Enunciado del Proyecto

4.1 Descripción del problema o necesidad a resolver

Markdown es uno de los formatos de texto ligero más usados en el desarrollo de software actual: documentación de repositorios (README), wikis, blogs técnicos y hasta este mismo enunciado se escriben con esta sintaxis. Su popularidad se debe a que combina legibilidad como texto plano con una traducción directa a HTML.

El problema que este proyecto busca resolver es la necesidad de comprender, más allá de usarlo, cómo se procesa formalmente un lenguaje con estructura anidada como Markdown: cómo una gramática libre de contexto captura la posibilidad de listas dentro de listas o citas dentro de citas, y cómo un parser descendente recursivo traduce esa gramática a código que construye un árbol de derivación y, a partir de él, genera un documento HTML equivalente, detectando y reportando con precisión cualquier error léxico o sintáctico que el documento de entrada pueda contener.

4.2 Alcance del proyecto

Alcance obligatorio:

- Subconjunto de Markdown: encabezados (# a #####), párrafos, texto en negrita (**texto**), texto en cursiva (*texto*), código en línea (`texto`), listas no ordenadas con al menos 3 niveles de anidamiento por indentación, citas (>) con al menos 2 niveles de anidamiento, bloques de código delimitados por ````` (contenido literal, no tokenizado), y regla horizontal (---).
- Analizador léxico implementado manualmente en Python con la clase `AnalizadorLexico` y el método `siguiente_token()`. Prohibido el uso del módulo `re` para la tokenización principal.
- Parser descendente recursivo implementado manualmente en Python con la clase `AnalizadorSintactico`, basado en la gramática libre de contexto de la sección 4.6. Prohibido el uso de librerías de parsing (`ply`, `lark`, `antlr4`, etc.) y de librerías de conversión Markdown (`markdown`, `mistune`, `python-markdown2`, etc.).
- Construcción de un árbol de derivación (AST) a partir del cual se genera el documento HTML de salida. No se permite generar el HTML por sustitución de texto o expresiones regulares.

- Tabla de tokens: número, lexema, tipo de token, línea y columna, con precisión del 100%.
- Tabla de errores léxicos y sintácticos: número, lexema, tipo de error, descripción y posición exacta. El análisis debe continuar tras cada error, sin detenerse (modo pánico).
- Tres reportes HTML con CSS embebido (descritos en la sección 4.8).
- Diagrama Graphviz (código DOT) con el árbol de derivación real resultante del análisis sintáctico.
- Interfaz gráfica en Python (se sugiere Tkinter, aunque puede usarse cualquier librería de GUI en Python) con todos los paneles requeridos.

Alcance opcional:

- Listas ordenadas (1. 2. 3.).
- Enlaces [texto](url) e imágenes ![alt](url).
- Tablas Markdown delimitadas con |.
- Texto tachado (~~texto~~) y listas de tareas (- [] / - [x]).
- Vista previa en vivo del HTML dentro de la misma interfaz gráfica.
- Advertencia visual cuando el documento excede una profundidad de anidamiento configurable.
- Resaltado de sintaxis (colorear palabras clave, cadenas, etc.) dentro de los bloques de código del Reporte 1, usando una librería externa como Prism.js o highlight.js. Esto es puramente visual y no debe alterar en nada el análisis léxico-sintáctico del documento Markdown ni la forma en que se reconoce el bloque de código como token literal.

4.3 Recursos y herramientas a utilizar

Tipo (Obligatorio / opcional)	Categoría (Software / hardware / Plataforma / Etc)	Descripción
Obligatorio	Software - Lenguaje	Python 3.10 o superior.
Obligatorio	Software - GUI	Tkinter (biblioteca estándar) u otra librería de interfaz gráfica en Python (PyQt, wxPython, etc.).
Obligatorio	Plataforma	Git y GitHub para control de versiones.
Obligatorio	Herramienta de diseño	Graphviz, para el árbol de derivación en el manual técnico.

4.4 Entregables

A continuación se detalla cada uno de los elementos que deberá presentar el estudiante

Tipo	Descripción										
Código Fuente	Código completo en Python, organizado en clases (Token, AnalizadorLexico, NodoAST, AnalizadorSintactico, GestorErrores, GeneradorHTML y la clase principal de la interfaz gráfica), debidamente comentado y siguiendo buenas prácticas de programación orientada a objetos.										
Manual Técnico (PDF)	Arquitectura del sistema, diagrama de clases, diagrama del AFD léxico con todos sus estados y transiciones, gramática libre de contexto completa, diagrama del árbol de derivación (Graphviz) de al menos un caso de prueba, descripción del algoritmo del parser descendente recursivo, estrategia de recuperación de errores, y justificación de decisiones de diseño.										
Manual de Usuario (PDF)	Guía paso a paso con capturas de pantalla: cómo ejecutar la aplicación, la sintaxis Markdown soportada, cómo cargar y analizar un archivo, cómo interpretar la tabla de tokens y el reporte de errores, y cómo abrir y navegar cada uno de los 3 reportes HTML generados.										
Casos de Prueba (PDF)	Documento con al menos 10 casos ejecutados: documento válido con todos los elementos obligatorios, listas con 3+ niveles de anidamiento, citas anidadas, negrita/cursiva sin cerrar, encabezado inválido, bloque de código sin cerrar, indentación inconsistente y casos borde. Cada caso con entrada, resultado esperado y resultado obtenido.										
Repositorio GitHub	Repositorio privado con el nombre LFP_<SECCIÓN>_2S2026, que contenga la carpeta del proyecto Proyecto2 con todo el código fuente en Python, así como un archivo README.md con las instrucciones de ejecución y ejemplos de uso. Agregar a su tutor al repositorio. <table border="1"> <thead> <tr> <th>Sección</th><th>Usuario Tutor</th></tr> </thead> <tbody> <tr> <td>A</td><td>Adriluuu</td></tr> <tr> <td>B+</td><td>SamuelAguilar18</td></tr> <tr> <td>B-</td><td>JoseOre12</td></tr> <tr> <td>N</td><td>JorgeP3</td></tr> </tbody> </table>	Sección	Usuario Tutor	A	Adriluuu	B+	SamuelAguilar18	B-	JoseOre12	N	JorgeP3
Sección	Usuario Tutor										
A	Adriluuu										
B+	SamuelAguilar18										
B-	JoseOre12										
N	JorgeP3										

4.5 Estructura del Subconjunto Markdown

El archivo de entrada con extensión .md es un documento de texto plano compuesto por bloques (encabezados, párrafos, listas, citas, bloques de código, reglas horizontales) que a su vez pueden contener elementos en línea (texto con negrita, cursiva o código). La siguiente tabla define los tipos de token que el AFD debe reconocer.

Token / Elemento	Tipo	Descripción / Regla	Ejemplo
NUMERAL (#)	Marcador de encabezado	De 1 a 6 símbolos # consecutivos al inicio de línea, seguidos de un espacio. Determina el nivel del encabezado (H1 a H6).	## Título
VIÑETA (-, *, +)	Marcador de ítem de lista	Uno de estos tres caracteres al inicio de línea (respetando la indentación), seguido de un espacio. No se confunde con NEGRITA/CURSIVA porque solo se evalúa como viñeta en posición inicial de línea.	- Elemento
INDENT / DEDENT	Token virtual de anidamiento	Generado por el analizador léxico al detectar un aumento o disminución del nivel de indentación (2 espacios = 1 nivel) entre ítems de lista consecutivos, de forma análoga al manejo de bloques de Python.	(no tiene lexema visible)
CITA (>)	Marcador de cita	Uno o más símbolos > consecutivos al inicio de línea. La cantidad de > indica el nivel de anidamiento de la cita.	>> Cita anidada
CERCA_CODIGO (```)	Delimitador de bloque de código	Tres acentos graves consecutivos al	```python

		inicio de línea. Abre y cierra un bloque; todo el contenido entre ambas ocurrencias se toma como texto literal, sin tokenizar su interior.	
REGLA_HORIZONTAL (---)	Separador de bloque	Tres o más guiones solos en una línea (sin otro contenido).	---
NEGRITA (**)	Delimitador de énfasis fuerte	Doble asterisco. Abre y cierra un tramo de texto en negrita. Tiene prioridad sobre CURSIVA (maximal munch: ** se reconoce antes que *).	**importante**
CURSIVA (*)	Delimitador de énfasis	Asterisco simple que no está en posición de viñeta de lista. Abre y cierra un tramo de texto en cursiva.	<i>*énfasis*</i>
CODIGO_INLINE (‘)	Delimitador de código en línea	Acento grave simple. Abre y cierra un tramo de texto tomado literalmente (sin interpretar negrita/cursiva dentro).	<code>`variable`</code>
TEXTO	Texto plano	Cualquier secuencia de caracteres que no coincide con ninguno de los delimitadores anteriores.	Lenguajes Formales
SALTO_PARRAFO	Separador de bloque	Una línea en blanco (solo espacios o vacía). Separa un	(línea vacía)

		bloque del siguiente.	
--	--	-----------------------	--

Reglas de prioridad y manejo especial:

- Viñeta vs. cursiva: -, * o + solo se reconocen como VIÑETA si aparecen al inicio de una línea (tras la indentación) seguidos de un espacio; en cualquier otra posición, * inicia el reconocimiento de CURSIVA.
- Negrita vs. cursiva: el AFD aplica la regla de mayor coincidencia (maximal munch): al encontrar **, siempre se reconoce como NEGRITA antes de evaluarlo como dos CURSIVA consecutivas.
- Encabezado vs. texto plano: los símbolos # solo forman un NUMERAL si están al inicio de línea y son seguidos de un espacio; "#sinespacio" es TEXTO plano, no un encabezado.
- Indentación: cada nivel de sublista equivale a 2 espacios adicionales de indentación respecto a su lista contenedora. Un salto de más de un nivel de una sola vez (por ejemplo, de 0 a 4 espacios sin pasar por 2) se reporta como error de indentación.
- Bloques de código: una vez reconocido CERCA_CODIGO de apertura, el analizador entra en modo literal y no vuelve a tokenizar NEGRITA, CURSIVA, VIÑETA, etc. hasta encontrar el CERCA_CODIGO de cierre.
- Citas anidadas: el número de símbolos > consecutivos al inicio de línea determina el nivel de anidamiento; una línea con > > (con espacio) o >> es equivalente y ambas notaciones deben aceptarse.
- Delimitadores sin cerrar: si se abre **, * o ` y el bloque actual (párrafo, ítem de lista, etc.) termina sin el cierre correspondiente, se reporta un error léxico en la posición de apertura.

4.6 Gramática Libre de Contexto

El estudiante debe definir formalmente la gramática libre de contexto (GLC) del subconjunto de Markdown soportado, e implementar un parser descendente recursivo que la respete. A continuación se presenta una gramática de referencia; el estudiante puede refinarla o extenderla, siempre que lo justifique en el manual técnico.

```
<documento>      ::= <bloque> <documento> | ε

<bloque>          ::= <encabezado> | <parrafo> | <lista> | <cita>
                  | <bloque_codigo> | <regla_horizontal>

<encabezado>      ::= NUMERAL <en_linea>

<parrafo>         ::= <en_linea> SALTO_PARRAFO

<lista>           ::= <item_lista> <lista> | <item_lista>

<item_lista>      ::= VIÑETA <en_linea> <sublista>
                  | VIÑETA <en_linea>

<sublista>        ::= INDENT <lista> DEDENT

<cita>            ::= CITA <bloque> <cita>
                  | CITA <bloque>
```

```
<bloque_codigo> ::= CERCA_CODIGO TEXTO_LITERAL CERCA_CODIGO

<regla_horizontal> ::= REGLA_HORIZONTAL

<en_linea> ::= <elemento_linea> <en_linea> | ε

<elemento_linea> ::= <negrita> | <cursiva> | <codigo_inline> | TEXTO

<negrita> ::= "***" <en_linea> "***"

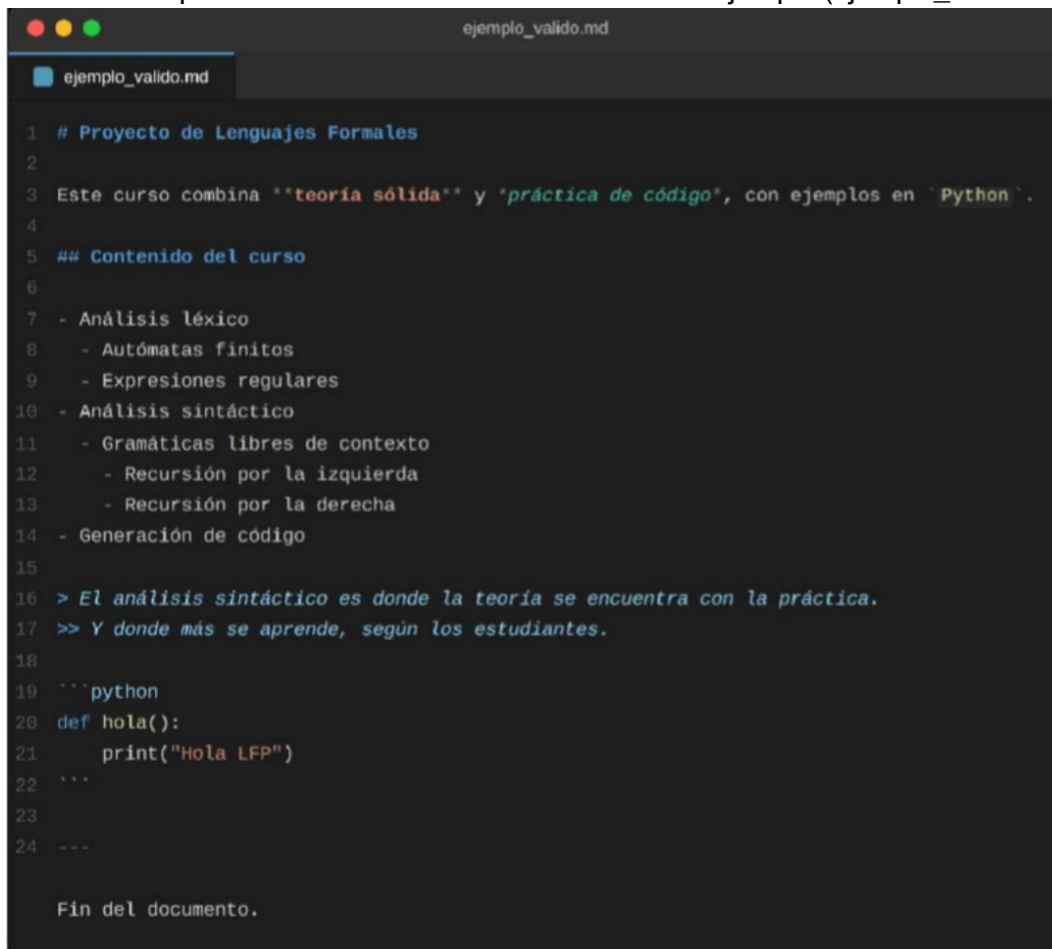
<cursiva> ::= "*" <en_linea> "*"

<codigo_inline> ::= "`" TEXTO_LITERAL "`"
```

Nota: esta gramática es una referencia mínima. Fíjate que <en_linea> y <negrita>/<cursiva> son mutuamente recursivas (una negrita puede contener cursiva y viceversa), y que <cita> es recursiva directa para soportar el anidamiento de citas. Esa recursión es exactamente lo que un parser descendente recursivo resuelve de forma natural: una función por cada símbolo no terminal, que se invoca a sí misma cuando la gramática lo indica.

4.7 Ejemplo de Archivo de Entrada

A continuación se presenta un archivo de entrada válido de ejemplo (ejemplo_valido.md):



```
ejemplo_valido.md

1 # Proyecto de Lenguajes Formales
2
3 Este curso combina teoría sólida y práctica de código, con ejemplos en Python.
4
5 ## Contenido del curso
6
7 - Análisis léxico
8   - Autómatas finitos
9   - Expresiones regulares
10  - Análisis sintáctico
11    - Gramáticas libres de contexto
12    - Recursión por la izquierda
13    - Recursión por la derecha
14  - Generación de código
15
16 > El análisis sintáctico es donde la teoría se encuentra con la práctica.
17 >> Y donde más se aprende, según los estudiantes.
18
19 ```python
20 def hola():
21     print("Hola LFP")
22 ```
23
24 ---

Fin del documento.
```

Este archivo tiene tres niveles de anidamiento en la lista ("Análisis sintáctico" → "Gramáticas libres de contexto" → "Recursión por la izquierda"), dos niveles de cita anidada, y combina negrita, cursiva y código en línea dentro del mismo párrafo — cubre, en un solo archivo, la mayoría de las reglas de la sección 4.5.

El Reporte 1 (sección 4.8) para este archivo debe producir HTML equivalente a, entre otras etiquetas: `<h1>Proyecto de Lenguajes Formales</h1>`, `<ul><li>Análisis léxico</li></ul></li></ul>` para la lista anidada, `<blockquote>...</blockquote>` para la cita anidada, y `<pre><code class="language-python">...</code></pre>` para el bloque de código.

Así se vería el Reporte 1 (documento HTML renderizado) al abrirlo en el navegador:



Nota: el bloque de código se muestra tal cual, sin colorear — colorear la sintaxis del código es parte del alcance opcional (sección 4.2).

4.8 Reportes a Generar

El sistema debe generar automáticamente tres reportes HTML al finalizar el análisis exitoso (sin errores), a partir del árbol de derivación construido por el parser. Cada reporte se

guarda como un archivo HTML independiente con CSS embebido y debe poder abrirse en cualquier navegador.

Reporte 1 — Documento HTML Renderizado

Objetivo: generar el documento HTML resultante de traducir el árbol de derivación completo, con las etiquetas semánticas correspondientes a cada elemento reconocido. Este reporte es la traducción propiamente dicha, y su correctitud depende directamente de que el parser haya construido bien el árbol.

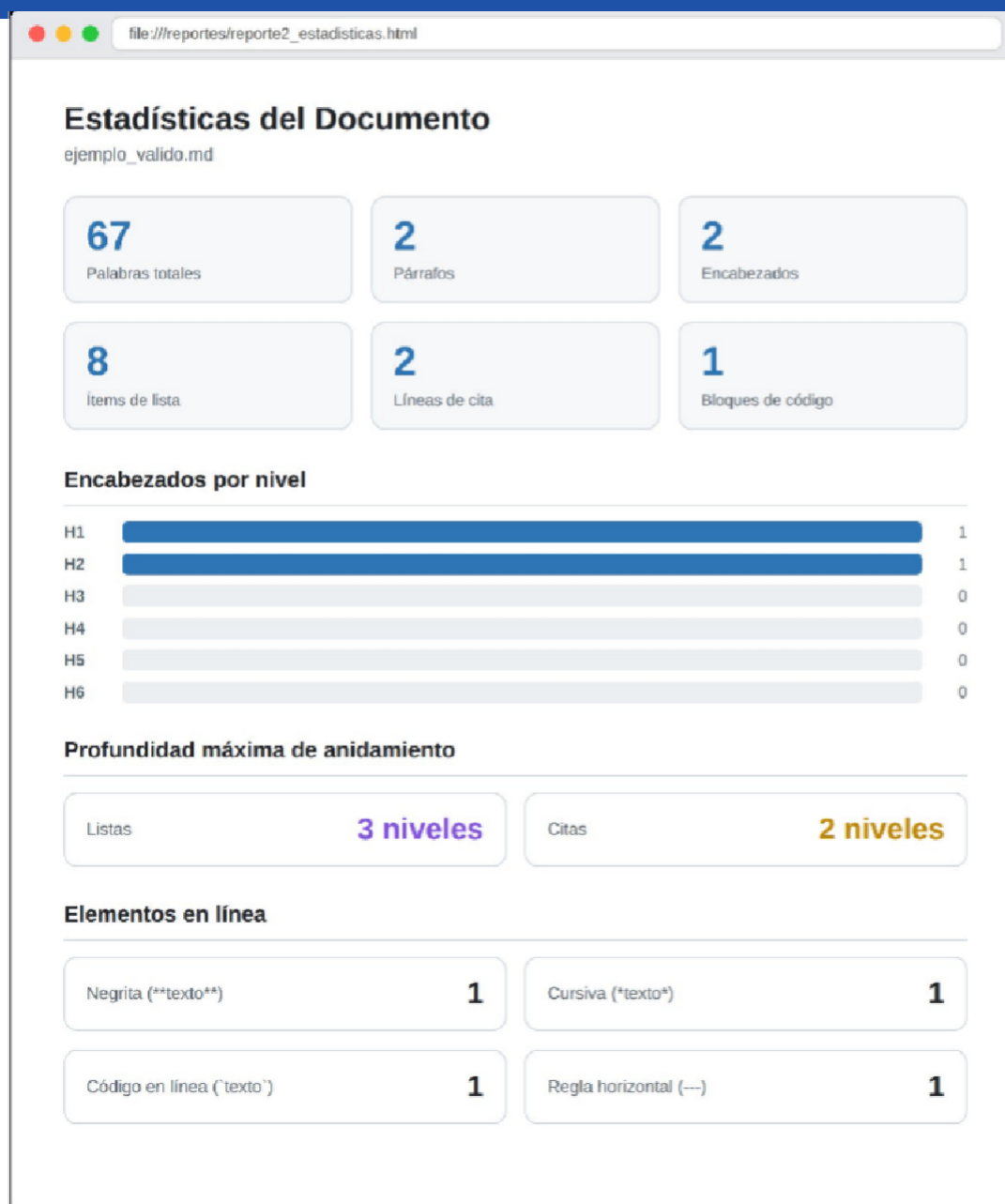
- Encabezados (# a #####) traducidos a <h1> a <h6>.
- Listas anidadas traducidas a con la jerarquía de anidamiento preservada exactamente.
- Citas anidadas traducidas a <blockquote> anidados en la misma profundidad que el original.
- Negrita, cursiva y código en línea traducidos a , y <code> respectivamente, incluyendo combinaciones (por ejemplo, cursiva dentro de negrita).
- Bloques de código traducidos a <pre><code>, preservando el contenido literal sin interpretar símbolos Markdown internos.
- Regla horizontal traducida a <hr>.

Reporte 2 — Estadísticas del Documento

Objetivo: mostrar un resumen cuantitativo de la estructura del documento analizado, útil para que el usuario entienda la composición del archivo sin tener que leerlo completo.

- Total de palabras y de párrafos.
- Cantidad de encabezados por nivel (H1 a H6), en una tabla o gráfico de barras simple.
- Cantidad de listas y profundidad máxima de anidamiento alcanzada.
- Cantidad de citas y profundidad máxima de anidamiento alcanzada.
- Cantidad de bloques de código, y cantidad de tramos en negrita, cursiva y código en línea.

Ejemplo del Reporte 2 para el archivo de la sección 4.7:



Reporte 3 — Tabla de Contenido / Mapa de Navegación

Objetivo: generar, a partir únicamente de los encabezados reconocidos por el parser, un índice tipo tabla de contenido con enlaces internos (anclas) hacia cada sección dentro del Reporte 1, respetando visualmente la jerarquía de niveles (un H3 se muestra más indentado que el H2 que lo contiene).

- Cada entrada del índice enlaza (con `<a href="#ancla">`) a la sección correspondiente en el Reporte 1.
- La indentación visual del índice refleja el nivel del encabezado (H1, H2, H3, ...).
- Si el documento no tiene encabezados, el reporte debe indicarlo explícitamente en vez de mostrarse vacío sin explicación.

Ejemplo del Reporte 3 para el archivo de la sección 4.7:



file:///reportes/reporte3_indice.html

Tabla de Contenido

ejemplo_valido.md — generado a partir de los encabezados reconocidos

- H1 Proyecto de Lenguajes Formales
 - H2 Contenido del curso

Cada entrada enlaza a su sección correspondiente dentro del Reporte 1. Con más niveles de encabezado (###, ####, ...) el índice agregaría más entradas, indentadas según su nivel.

4.9 Tabla de Errores Léxicos y Sintácticos

Cuando el archivo .md contiene elementos inválidos, el analizador debe acumular todos los errores durante el recorrido completo del archivo (modo de recuperación de errores) y mostrarlos al finalizar, tanto en la interfaz gráfica como en una tabla dedicada. El sistema no debe detenerse ante el primer error; debe continuar el análisis para reportar todos los problemas en una sola pasada.

Tipo de Error	Fase	Descripción y mensaje sugerido
NEGRITA_SIN_CERRAR	Léxico	Se abrió ** y el bloque actual terminó sin el cierre correspondiente. Mensaje: "Negrita sin cerrar iniciada en línea L, columna C".
CURSIVA_SIN_CERRAR	Léxico	Se abrió * (en posición de énfasis, no de viñeta) y el bloque actual terminó sin cierre. Mensaje: "Cursiva sin cerrar iniciada en línea L, columna C".
CODIGO_INLINE_SIN_CERRAR	Léxico	Se abrió ` y el bloque actual terminó sin el cierre correspondiente. Mensaje: "Código en línea sin cerrar iniciado en línea L, columna C".
BLOQUE_CODIGO_SIN_CERRAR	Léxico	Se abrió ``` y se alcanzó el fin de archivo sin el cierre correspondiente. Mensaje: "Bloque de código sin cerrar

		iniciado en línea L".
ENCABEZADO_INVALIDO	Léxico	Más de 6 símbolos # consecutivos al inicio de línea. Mensaje: "Encabezado inválido en línea L: máximo 6 niveles".
INDENTACION_INVALIDA	Sintáctico	Un ítem de lista aumenta su indentación en más de un nivel respecto al ítem anterior (por ejemplo, de 0 a 4 espacios sin pasar por 2). Mensaje: "Indentación inválida en línea L, columna C".
CITA_MAL_FORMADA	Sintáctico	Un símbolo > no está seguido de un espacio o de otro >. Mensaje: "Cita mal formada en línea L, columna C".

4.10 Árbol de Derivación con Graphviz

Adicionalmente a los tres reportes HTML, el sistema debe generar un archivo arbol.dot con código Graphviz que represente el árbol de derivación real resultante del análisis sintáctico del archivo cargado. Este árbol muestra cómo las producciones de la gramática de la sección 4.6 se aplicaron para reconocer la estructura del documento.

- Un nodo raíz con el símbolo inicial de la gramática (<documento>).
- Nodos internos por cada producción de la gramática aplicada (uno por cada <bloque>, <lista>, <item_lista>, <cita>, <en_linea>, etc. reconocido).
- Nodos hoja por cada token terminal reconocido (NUMERAL, TEXTO, VIÑETA, etc.).
- Los nodos terminales deben tener un color distinto a los no terminales.

Código DOT de ejemplo (parcial):

```
digraph ArbolDerivacion {
    rankdir=TB;
    node [shape=box, style=filled, fontname="Arial"];

    n0 [label="<documento>", fillcolor="#2E75B6", fontcolor=white];
    n1 [label="<bloque>", fillcolor="#D6EAF8"];
    n2 [label="<encabezado>", fillcolor="#D6EAF8"];
    n3 [label="NUMERAL", fillcolor="#F9E79F"];
    n4 [label="<en_linea>", fillcolor="#D6EAF8"];
    n5 [label="TEXTO", fillcolor="#F9E79F"];

    n0 -> n1; n1 -> n2; n2 -> n3; n2 -> n4; n4 -> n5;
    // ... continua con el resto del arbol
}
```

5. Material de apoyo

Categoría (Manual / Documentación oficial/ Ejemplo / Etc)	Link	Descripción
Manual	N/A	Compilers: Principles, Techniques, and Tools (Dragon Book) — Aho, Sethi, Ullman. Capítulos 2, 3 y 4 (parsing descendente recursivo).
Manual	N/A	Crafting Interpreters — Robert Nystrom, capítulos sobre parsing (disponible gratis en línea).
Documentación oficial	https://www.markdownguide.org/basic-syntax/	Referencia oficial de la sintaxis básica de Markdown.
Documentación oficial	https://graphviz.org/doc/info/lang.html	Referencia del lenguaje DOT de Graphviz.

**El link es opcional para una mejor guía del material de apoyo

6. Metodología

A continuación se presenta una metodología de la lógica que puedes seguir para elaborar el presente proyecto:

Pasos a seguir:

1. Fase 1: Diseño de la Gramática y del AFD Léxico (Semana 1)
Definir formalmente la gramática libre de contexto del subconjunto de Markdown (sección 4.6) y el alfabeto/tokens del analizador léxico (sección 4.5). Diseñar el diagrama del AFD léxico con todos sus estados y transiciones, y la arquitectura general del sistema (clases y responsabilidades).
2. Fase 2: Implementación del Analizador Léxico (Semana 2)
Implementar la clase Token y la clase AnalizadorLexico con siguiente_token(), incluyendo el manejo de INDENT/DEDENT para las listas anidadas y el modo literal para los bloques de código. Verificar el funcionamiento desde consola con los casos de prueba léxicos.
3. Fase 3: Implementación del Parser y de la GUI (Semana 3)
Implementar la clase AnalizadorSintactico como parser descendente recursivo (una función por cada símbolo no terminal de la gramática), construyendo el árbol de

derivación (AST). Diseñar e integrar la interfaz gráfica en Python con todos los paneles requeridos.

4. Fase 4: Generación de Reportes, Pruebas y Documentación (Semana 4)
Implementar el GeneradorHTML que recorre el AST para producir el Reporte 1, y los generadores de los Reportes 2 y 3, así como el archivo arbol.dot. Ejecutar los casos de prueba, corregir errores, y redactar el Manual Técnico y el Manual de Usuario.

Recomendaciones

No intentes escribir primero el parser completo y probarlo hasta el final: implementa y prueba una producción de la gramática a la vez (primero encabezados y párrafos, luego listas simples, luego listas anidadas, luego citas). Cada nivel de anidamiento que agregues debe seguir pasando las pruebas de los niveles anteriores.

7. Cronograma

Tipo	Fecha Inicio	Fecha Fin
Asignación de Proyecto	24/09/2026	24/09/2026
Elaboración	24/09/2026	20/10/2026
Calificación	24/10/2026	24/10/2026

8. Rúbrica de Calificación

En esta sección se aborda todo lo relacionado a la calificación, lo que permite claridad para el estudiante y el tutor académico de los aspectos a evaluar y los mismos que conocen desde el inicio por lo que todo proyecto debe incluirlos desde el inicio.

8.1 Requisitos para optar a la calificación

Antes de la evaluación del proyecto, debe cumplir con los requisitos que se indiquen en esta sección, de no cumplir usted tendrá una nota con valor de cero puntos.

Tema	Descripción	Cumple (Sí/No)
------	-------------	----------------

Tecnología establecida	El desarrollo debe realizarse en Python 3.10+ con interfaz gráfica en Python (Tkinter u otra librería de GUI). No se permite el uso del módulo re para la tokenización principal, ni librerías de parsing (ply, lark, antlr4, etc.) ni librerías de conversión Markdown (markdown, mistune, python-markdown2, etc.).	
Herramientas requeridas	El proyecto debe incluir control de versiones con Git, repositorio en GitHub con commits organizados y README completo.	
Documentación obligatoria	Manual Técnico, Manual de Usuario y Casos de Prueba en formato PDF.	
Funcionalidad mínima	Cargar archivos .md, tokenizar mediante AFD, parsear mediante un parser descendente recursivo construyendo un árbol de derivación, generar tabla de tokens y tabla de errores con posición exacta, generar los 3 reportes HTML y el árbol de derivación en Graphviz.	

8.2 Detalle de la Calificación

	Criterio/Nivel			Nivel de evaluación		
No.	Criterio de evaluación	Punteo maximo		Satisfactorio 100% - 61%	Necesita mejorar 60% - 0%	Punteo Obtenido
1	Habilidades	40				
1.1	Documentación Técnica (Manual Técnico)	10	Descripción	El manual técnico está completo y profesional. Incluye la gramática libre de contexto completa, el diagrama del AFD léxico, el diagrama del árbol de derivación (Graphviz), la arquitectura del parser descendente recursivo, la estrategia de recuperación de errores y justificación sólida de las decisiones de diseño.	El manual técnico está incompleto, carece de la gramática formal o del diagrama del AFD/árbol de derivación, o la presentación es desorganizada.	
			Rango del punteo	10-6/	5-0/	
1.2	Documentación de Usuario (Manual de Usuario)	8	Descripción	El manual de usuario es completo, claro y profesional. Incluye instrucciones de ejecución, capturas de pantalla de la GUI	El manual de usuario está incompleto, sin capturas de los reportes, o las instrucciones son	

				y de los 3 reportes generados, y la sintaxis Markdown soportada.	confusas o insuficientes.	
			Rango del punteo	8-5/	4-0/	
1.3	Repositorio GitHub	8	Descripción	El repositorio está bien organizado, con commits frecuentes y descriptivos, README completo con instrucciones de ejecución y .gitignore apropiado para Python.	El repositorio tiene pocos commits o poco descriptivos, README incompleto o ausente, o falta .gitignore.	
			Rango del punteo	8-5/	4-0/	
1.4	Casos de Prueba y Documentación de Testing	8	Descripción	Incluye al menos 10 casos de prueba bien documentados que cubren documentos válidos con anidamiento profundo, distintos errores léxicos y sintácticos, y casos borde, cada uno con entrada, resultado esperado y obtenido.	Los casos de prueba son insuficientes (menos de 10), están mal documentados, o no cubren anidamiento ni escenarios de error.	
			Rango del punteo	8-5/	4-0/	

1.5	Presentación	6	Descripción	La presentación es profesional, clara y bien estructurada, cubriendo introducción, demostración en vivo y explicación técnica de la gramática y del parser.	La presentación está incompleta, sin demostración funcional o sin explicación técnica del parser.	
			Rango del punteo	6-4/	3-0/	
	Sub-Total de Puntos					
2	Conocimientos	60				
2.1	Carga y Procesamiento de Archivos .md	5	Descripción	Carga correctamente archivos .md, valida que el archivo exista y sea legible, y actualiza los contadores de línea/columna con precisión.	No carga archivos correctamente o los contadores de línea/columna son incorrectos con frecuencia.	
			Rango del punteo	5-3/	2-0/	
2.2	Reconocimiento de Tokens mediante AFD	10	Descripción	El AFD implementado manualmente reconoce al menos el 90% de los tokens en los casos de prueba, incluyendo el manejo correcto de INDENT/DEDENT y el modo literal de los bloques de código. No utiliza el módulo re.	Reconoce menos del 75% de los tokens, no distingue correctamente casos ambiguos (viñeta vs. cursiva), o utiliza re para tokenizar.	

			Rango del punteo	10-6/	5-0/	
2.3	Tabla de Tokens	5	Descripción	Genera correctamente los campos número, lexema, tipo, línea y columna para todos los tokens reconocidos, visualizados en la interfaz gráfica con precisión del 100%.	Faltan campos, las posiciones son incorrectas con frecuencia, o la tabla no se muestra correctamente en la interfaz.	
			Rango del punteo	5-3/	2-0/	
2.4	Detección y Reporte de Errores Léxicos y Sintácticos	8	Descripción	Detecta las categorías de error descritas en la sección 4.9, reporta la posición exacta y un mensaje descriptivo, y continúa el análisis tras cada error (recuperación de errores).	No detecta alguna categoría de error, las posiciones son incorrectas, o el análisis se detiene ante el primer error.	
			Rango del punteo	8-5/	4-0/	
2.5	Implementación del Parser Descendente Recursivo	15	Descripción	El parser implementa correctamente la gramática de la sección 4.6 mediante una función por cada símbolo no terminal, construyendo un árbol de derivación (AST) que refleja fielmente el anidamiento de listas (3+ niveles) y citas (2+ niveles) del documento analizado.	El parser no maneja correctamente la recursión de la gramática, no soporta múltiples niveles de anidamiento, construye un árbol incompleto o incorrecto, o no sigue la gramática definida en el manual técnico.	

			Rango del punteo	15-9/	8-0/	
2.6	Generación del HTML de Salida a partir del AST	7	Descripción	El HTML generado recorre el árbol de derivación (no el texto original) y refleja correctamente la jerarquía de listas y citas anidadas, así como negrita, cursiva y código en línea, incluyendo combinaciones entre ellos.	El HTML se genera por sustitución de texto/expresiones regulares en vez de recorrer el AST, o no refleja correctamente el anidamiento.	
			Rango del punteo	7-4/	3-0/	
2.7	Árbol de Derivación en Graphviz	5	Descripción	El archivo arbol.dot generado representa fielmente el árbol de derivación real del documento analizado, con nodos terminales y no terminales diferenciados por color, según la sección 4.10.	El árbol generado no corresponde al análisis real, está incompleto, o no diferencia visualmente los nodos terminales de los no terminales.	
			Rango del punteo	5-3/	2-0/	
2.8	Interfaz Gráfica	5	Descripción	La interfaz gráfica en Python es funcional e intuitiva: permite cargar el archivo, ejecutar el análisis, y visualizar la tabla de tokens, la tabla de errores y el acceso a los 3 reportes.	Faltan componentes de la interfaz, los controles no funcionan correctamente, o no se pueden visualizar los resultados del análisis.	
			Rango del punteo	5-3/	2-0/	

	Sub-Total de Puntos					
	Total					

8.3 Comentarios Generales
